

A decision tree is a machine learning model used for classification and regression. It has a tree structure where internal nodes make decisions based on features, branches show outcomes, and leaf nodes give the final prediction.

Key Components of a Decision Tree

- **Root Node:** Top node representing the entire dataset; first split occurs here.
- **Internal Nodes:** Decision nodes that test feature values.
- **Branches:** Paths showing outcomes of decisions.
- **Leaf Nodes:** Terminal nodes that provide the final class or value.

How Decision Trees Work

Decision trees recursively split the dataset using the feature that best separates the target.

1. **Selecting the Best Split:** At each node, the tree selects the best feature to split the data using criteria such as:
 - a. **Gini Impurity:** Measures node impurity (used in CART).
 - b. **Entropy:** Measures disorder (used in ID3, C4.5).
 - c. **Information Gain:** Reduction in entropy after split.
 - d. **Mean Squared Error (MSE):** Used for regression.
2. **Splitting the Dataset:** Divide data into subsets based on the chosen feature.
3. **Recursion:** Repeat the process for each subset to form subtrees.
4. **Stopping Criteria:** Stop when:
 - a. Node contains a single class.
 - b. Maximum depth is reached.
 - c. Further splits give little improvement.

1 Why are decision trees non-parametric models?

Parametric models make strong assumptions about the functional form, or shape, of the relationship between the variables in the data. (e.g., linear regression assumes a linear relationship). The model parameters are fixed in number and are typically estimated using techniques like maximum likelihood estimation or least squares.

Non-parametric models, in contrast, make fewer assumptions about the functional form of the relationship between variables. Instead of a fixed set of parameters, these models typically have parameters that grow with the size of the data, allowing them to be more flexible in modeling complex relationships. **Decision trees** are called **non-parametric models** because:

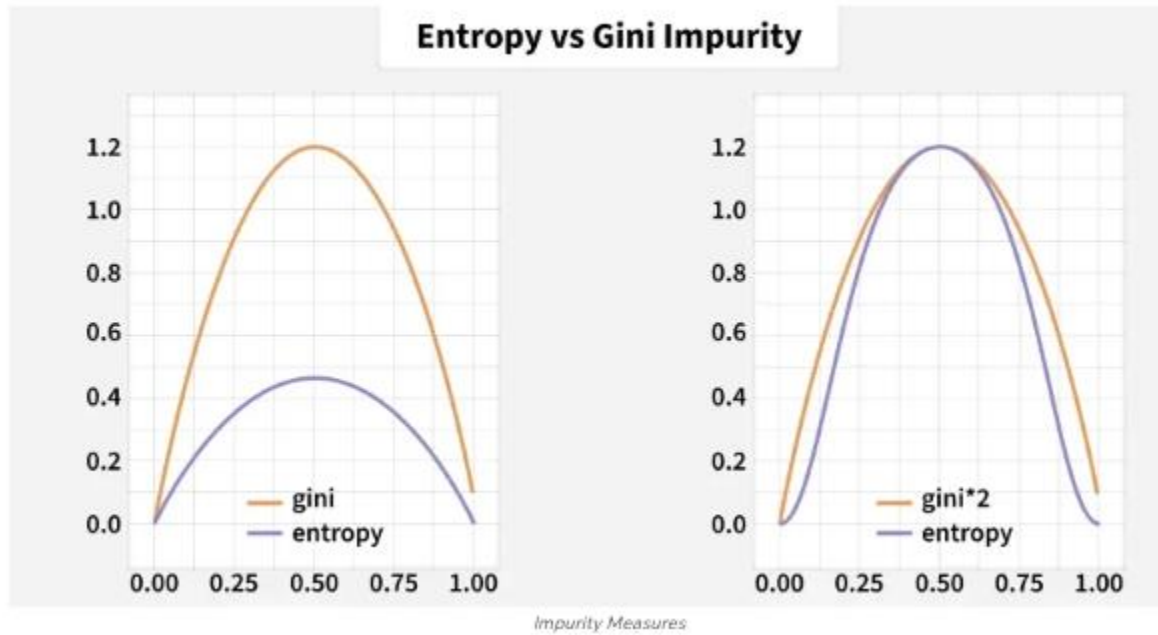
- They do not assume any fixed functional form for the relationship between input features and the target variable
- The number of model parameters is not fixed in advance but grows with the data
- Their model complexity (number of splits/nodes) grows adaptively with the training data.

2 Compare entropy vs Gini mathematically.

Decision Tree are classification models that split data into nodes based on feature values. Gini Impurity and Entropy are two measures used in decision trees to decide how to split data into branches. These measures help avoid random splits, improve classification accuracy, and keep the model more interpretable and consistent. Gini Index has values inside the interval $[0, 0.5]$ whereas the interval of the Entropy is $[0, 1]$.

Let a node contain class probabilities

$$p_1, p_2, \dots, p_K, \quad \sum_{i=1}^K p_i = 1$$



Criterion	Gini Index	Entropy
Definition	Measures impurity based on squared probabilities	Measures impurity using log probabilities
Formula	$1 - \sum p_i^2$	$-\sum p_i \log_2 p_i$
Range	[0, 0.5] for binary classification	[0, 1] for binary classification
Computation	Faster (no logarithm)	Slower (uses logarithm)
Decision Tree Preference	CART (Classification and Regression Trees) uses Gini by default	ID3, C4.5 use Entropy

Gini measures impurity using squared probabilities $1 - \sum p_i^2$, while entropy measures uncertainty using logarithmic information $-\sum p_i \log p_i$; both are minimized to obtain purer decision tree splits.

3 Why are trees unstable?

- Decision trees have high variance, so even small changes in the training data can lead to different best splits and a different tree structure.
- The greedy split selection means the tree chooses the split that gives the maximum immediate impurity reduction at each node without considering future splits, so an early different choice can propagate large changes throughout the tree.
- Trees use axis-aligned threshold splits, where each decision boundary is of the form $x_j \leq t_{x_j} \ \forall j$ (splitting on one feature at a time), so samples near the threshold can easily switch sides due to small noise.
- Deep trees tend to overfit the training data, making the model highly sensitive to sampling variations and therefore unstable.

4 What is the computational complexity of training?

Computational complexity is a measure of the amount of computing resources (time and space) that a particular algorithm consumes when it runs. Computational complexity is further divided into two types of complexities which are:

Time complexity Rather than measuring the time taken by an algorithm or a piece of code that introduces machine dependency, Time complexity was created. Time complexity is a function of input that eliminates machine dependency and allows us to compare different algorithms without even running them.

Space complexity *The space complexity of an algorithm or a computer program is the amount of memory space required to solve an instance of the computational problem as a function of characteristics of the input.*

Decision Trees:

n = Number of training examples

f = no of features

d = depth of the tree

p = no of nodes

Training Time complexity: $O(n \log(n) * f)$

Prediction Time complexity: $O(d)$

Runtime space complexity: $O(p)$

Random Forests:

n = Number of training examples

f = no of features

k = no of trees

p = no of nodes in trees

d = depth of the tree

Training Time complexity: $O(n \log(n) * f * k)$

Prediction Time complexity: $O(d * k)$

Runtime space complexity: $O(p * k)$

5 Prove that greedy splitting does not guarantee global optimum.

Let **IG** denote Information Gain and **H** denote entropy.

XOR (Exclusive OR) is a logical function where the output is 1 if the two inputs are different and 0 if they are the same. Mathematically,

$$y = x_1 \oplus x_2$$

Consider the XOR dataset:

x_1	x_2	y
0	0	0
0	1	1
1	0	1
1	1	0

At the root:

$$p(y = 0) = p(y = 1) = 0.5$$

$$H(\text{root}) = 1$$

If we split on x_1 :

Left node ($x_1 = 0$) has classes $\{0, 1\}$

$$H_L = 1$$

Right node ($x_1 = 1$) has classes $\{1, 0\}$

$$H_R = 1$$

Weighted entropy after split:

$$H_{\text{after}} = 1$$

$$IG(x_1) = H(\text{root}) - H_{\text{after}} = 1 - 1 = 0$$

Similarly,

$$IG(x_2) = 0$$

Greedy splitting chooses the feature with maximum IG, but here no split improves impurity, so the greedy method stops.

However, if we allow a depth-2 tree (first split on x_1 , then on x_2), each leaf becomes pure:

$$H_{\text{leaves}} = 0$$

which is the global optimum.

Therefore, greedy splitting maximizes only immediate IG and does not consider future splits, so it does not guarantee the globally optimal decision tree. 

6 How does tree depth relate to VC dimension?

VC Dimension (VC) is the maximum number of data points that a model can shatter (i.e., classify correctly in all possible labelings), and it measures the model's capacity or complexity.

For a binary decision tree of depth d :

$$L \leq 2^d, \quad VC = O(L \log L) \approx O(2^d \cdot d)$$

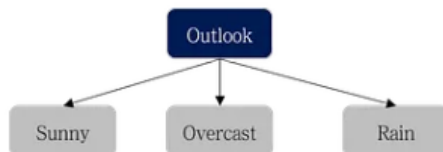
Key implication: As tree depth increases, the VC dimension grows exponentially, which increases model capacity and the risk of overfitting, while shallow trees have lower VC and may underfit.

7 Why do trees overfit small datasets?

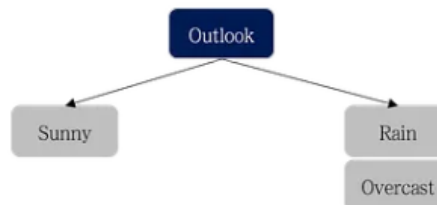
- Decision trees have high variance and can easily memorize small datasets, fitting noise instead of the true underlying pattern.
- With few samples, impurity measures (like Gini or entropy) become unreliable, so the tree makes splits based on random fluctuations.
- Deep trees keep splitting until leaves are nearly pure, which causes the model to fit training points exactly rather than generalize.
- Small datasets provide poor coverage of the feature space, so the learned decision boundaries become overly specific and fail on new data.

8 Compare CART vs ID3.

ID3



CART



CART (Classification and Regression Trees) and ID3 (Iterative Dichotomiser 3) are two fundamental algorithms for creating decision trees in machine learning. They are used to split datasets into subsets based on attribute values, eventually forming a tree structure that can be used for classification or regression.

Key Characteristics of ID3

- **Uses Entropy & Information Gain:** Selects splits using entropy and information gain.
- **Categorical Data:** Designed mainly for categorical features; continuous data must be discretized.
- **Greedy Approach:** Top-down recursive splitting based on highest information gain.

Steps of ID3

1. **Calculate Entropy:** Compute entropy of the target class.
2. **Calculate Information Gain:** Measure entropy reduction for each feature.
3. **Select Best Attribute:** *Choose the attribute with the highest information gain to split the dataset.*
4. **Create Subtrees:** *Split the dataset into subsets using the selected attribute and repeat the process recursively for each subset.*
5. **Stopping Criteria:** *The recursion stops when all data points belong to a single class, or there are no more attributes to split on.*

CART stands for Classification and Regression Trees and is a more advanced algorithm that can handle both classification and regression tasks.

Key Characteristics of CART

- **Uses Gini or MSE:** Gini impurity for classification; MSE for regression.

- **Handles Numerical & Categorical Data:** No need for prior discretization.
- **Binary Trees:** Each split creates exactly two child nodes.
- **Binary Trees:** Each split creates exactly two child nodes.

Steps of CART

1. **Select Best Split:** Choose split with maximum impurity reduction.
2. **Split Dataset:** Partition data into two subsets.
3. **Create Subtrees:** Recursively grow the tree.
4. **Pruning:** Remove weak branches to reduce overfitting.
5. **Stopping Criteria:** Stop when improvement is minimal or depth limit is reached.

Differences Between ID3 and CART

- **Data Types:** ID3 → mainly categorical; CART → categorical + numerical.
- **Splitting Criterion:** ID3 → information gain (entropy); CART → Gini (classification) / MSE (regression).
- **Tree Structure:** ID3 → multi-way splits; CART → binary splits.
- **Pruning:** CART → built-in pruning; ID3 → no inherent pruning.

9 Derive pruning as regularization.

Mathematical View

In decision trees, we minimize:

$$\text{Total Cost} = \text{Training Error} + \alpha \times \text{Tree Complexity}$$

$$R_{\alpha}(T) = R(T) + \alpha|T|$$

Meaning of Each Term

- T (**Tree**): The decision tree model.
- $R(T)$ (**Training Error**): Misclassification error (classification) or MSE (regression) of tree T .
- $|T|$ (**Tree Complexity**): Number of leaf nodes in the tree.
- α (**Regularization Parameter**): Controls penalty strength for complexity.
- $R_{\alpha}(T)$: Regularized cost function.

Interpretation

- Large tree $\rightarrow$ low training error but high complexity $\rightarrow$ overfitting
- Small tree $\rightarrow$ slightly higher training error but better generalization
- Pruning removes branches that do not reduce the regularized cost

Why Pruning = Regularization

- Regularization adds a penalty for model complexity
- Pruning removes unnecessary nodes (reduces $|T|$)
- Both aim to improve generalization

10 Why are trees invariant to monotonic transformations?

Decision trees are invariant to monotonic transformations because they make decisions based only on the order (ranking) of feature values, not their exact numerical magnitudes. A monotonic transformation is a function that preserves the order of data points (for example, log, square, or linear scaling with positive slope). When a decision tree chooses a split, it looks for a threshold that best separates the data. If the feature values are transformed monotonically, their relative ordering remains the same, so the same split point (in terms of data partition) will be found, even though the numeric threshold value changes. As a result, the structure of the tree and the predictions remain unchanged. Hence, scaling or applying any monotonic transformation to features does not affect decision tree performance.